BCSE302L - DATABASE SYSTEMS

Fall Semester 2026-2027

Case Study using MongoDB — D2 Slot

NEWS PORTAL

A document-oriented database for articles, authors and readers

Name	AHMED HADI JAFEER KAPPUMUGATH
Registration number	24BCE2743
Case study topic	News Portal
Database	newsPortalDB
Collections	categories, authors, articles, readers, comments

1. Case study understanding

The problem

A news portal is a website that publishes articles and lets registered readers read, bookmark and comment on them. The editorial side of it is messier than it looks from the front page. One article has a cover photo and a caption, the next one is a two-paragraph market update with no image at all, and a third is still a draft with no publish date. Tags differ from article to article, some stories are filed under one category and later moved, and a few are written by one reporter but edited by another.

A relational design handles this with a wide articles table full of nullable columns, plus a separate tags table and a separate media table that have to be joined back on every page load. For a site whose traffic is almost entirely reads, that join cost shows up on the homepage. MongoDB stores each article as a single document, so the fields that vary simply exist or do not exist, the tag list sits inside the article as an array, and the cover image and its caption sit inside the article as a sub-document. One query returns the whole article.

The portal in this case study is modelled on an Indian general-news site with eight desks, eight staff writers, a set of registered readers on free and premium plans, and a comment section that needs moderation.

Who uses the system

Authors write and submit articles, and each of them has a profile page showing a bio, areas of expertise and social links. Editors move an article from draft to published and decide what gets featured on the homepage. Readers register, pick the categories they want to follow, bookmark articles and post comments; a premium reader has a subscription with a start and an end date, a free reader does not. Moderators read the comment queue and remove spam. Finally there is the portal itself, which needs view counts, like counts and per-category totals to decide what to put in the trending box.

What the data has to support

Working backwards from those users, the database has to store article text together with its metadata, keep author profiles separate from the articles so that a profile edit does not touch two hundred documents, hold a category list that the editorial team controls, record each reader's preferences and subscription, and store comments with their replies. It also has to answer the queries the site runs constantly: the latest articles in a category, the three most-read stories, articles matching a search box, a reader's bookmarks, and the comment thread under a given article.

Two counters, views and likes, are updated far more often than anything else on the site. MongoDB's $inc operator updates them in place without reading the document first, which is the main reason they live on the article document rather than in a separate statistics collection.

2. Collection identification

The case study required at least five collections, three of which were given. The three supplied were articles, authors and categories. Two more were needed to cover the reader side of the portal, so readers and comments were added.

Collection	What it holds and why it is separate
`categories`	The eight editorial desks: Politics, Technology, Sports, Business, Health, Entertainment, Science and Education. A category is a small, stable record that hundreds of articles point at. Storing the name inside every article would mean rewriting every one of them to fix a spelling, so it gets its own collection.
`authors`	Staff writer profiles with a bio, an expertise list, social handles, a verification flag and a running article count. An author writes many articles and has a profile page of their own, which makes this a separate collection rather than an embedded block.
`articles`	The main collection. Each document is one story with its title, slug, summary, body text, tags, status, publish date, view and like counters, cover image and the city the story is filed from.
`readers`	Registered users. Each document carries the reader's city, the categories they follow, their newsletter flag, their subscription plan with its dates, and the articles they have bookmarked.
`comments`	Reader comments on articles, each with its like count, a moderation flag and an array of replies. Comments are high-volume and keep arriving long after an article is published, so they are kept out of the article document.

Why comments are not inside articles

Embedding comments in the article document was considered and rejected. A popular article can collect thousands of comments, and MongoDB caps a single document at 16 MB; more importantly, the homepage loads articles without their comments, so embedding would drag unwanted data into every read. Keeping comments separate also lets a moderator run deleteMany on flagged comments across the whole site in one operation, which is exactly what Task 8 does.

3. Document design

Field names, data types and the embed-or-reference decision for each collection are given below. The _id values are readable strings such as ART01 and AUT03 instead of the default ObjectId. This is deliberate: ObjectIds are 24 hex characters and make the query output in a screenshot almost unreadable, whereas ART03 can be followed by eye across a $lookup result.

categories

Field	Type	Description
_id	String	Category code, e.g. CAT02
name	String	Display name shown on the navigation bar
slug	String	URL fragment, e.g. /technology
description	String	One-line description of the desk
isActive	Boolean	False for a desk that has been retired
createdAt	Date	When the desk was added

authors

Field	Type	Description
_id	String	Author code, e.g. AUT04
name	String	Full name as it appears on the byline
email	String	Work email address
bio	String	Short biography for the profile page
expertise	Array of String	Beats the author covers, queried directly with an equality match
social	Embedded document	twitter and linkedin handles, either of which may be null
isVerified	Boolean	Whether the profile carries a verified badge
joinedOn	Date	Date the author joined the portal
articleCount	Number	Running total of published articles

articles

Field	Type	Description
_id	String	Article code, e.g. ART07
title	String	Headline
slug	String	URL fragment
summary	String	Standfirst shown under the headline
content	String	Body text
authorId	String (reference)	Points at authors._id

categoryId	String (reference)	Points at categories._id
tags	Array of String	Free-form tags used by the search and related-stories box
status	String	draft or published
publishedAt	Date or null	Null while the article is a draft
views	Number	Incremented on every page view
likes	Number	Incremented when a reader likes the story
readTimeMin	Number	Estimated reading time in minutes
isFeatured	Boolean	Whether the story sits in the homepage featured slot
media	Embedded document	coverImage filename and its caption
location	Embedded document	city and country the story is filed from

readers

Field	Type	Description
_id	String	Reader code, e.g. RDR05
name	String	Reader's name
email	String	Login email
city	String	City, used for local-news targeting
preferences	Embedded document	categories array of category codes, plus a newsletter boolean
subscription	Embedded document	plan, startDate and endDate; endDate is null on a free plan
registeredOn	Date	Sign-up date
bookmarks	Array of String (references)	Article codes the reader has saved

comments

Field	Type	Description
_id	String	Comment code, e.g. CMT06
articleId	String (reference)	Points at articles._id
readerId	String (reference)	Points at readers._id

`text`	String	The comment body
`postedAt`	Date	Timestamp, stored with time of day as well as date
`likes`	Number	Upvotes on the comment
`isFlagged`	Boolean	Set by the spam filter or by a moderator
`replies`	Array of embedded documents	Each reply has readerId, text and postedAt

Embedding against referencing

Four blocks of data are embedded. An article's media and location are embedded because they belong to that one article, are never queried on their own, and are always needed when the article is read. An author's social handles are embedded for the same reason. A reader's preferences and subscription are embedded because they are read and written as a unit whenever that reader logs in. Replies are embedded inside their parent comment because a reply is only ever shown underneath the comment it answers, and the number of replies on a single comment stays small.

Three relationships are referenced instead. An article stores authorId rather than a copy of the author, because an author writes dozens of articles and a bio edit must not require touching all of them. The same argument applies to categoryId. A comment stores both articleId and readerId, and a reader stores an array of article codes under bookmarks, because an article and a reader both exist independently of any bookmark.

The cost of referencing is that the join has to be done at query time. R11 and R12 in Task 6 do exactly that with the $lookup aggregation stage, pulling the author name and the category name back onto each article.

4. Database creation

MongoDB creates a database lazily: the use command switches context, but nothing is written to disk until the first document or an explicit createCollection call. The five collections were therefore created up front so that show collections would list all of them before any insert ran.

Creating the database and the five collections

```
use newsPortalDB

db.createCollection("categories")
db.createCollection("authors")
db.createCollection("articles")
db.createCollection("readers")
db.createCollection("comments")

show collections
```

use newsPortalDB switches the shell to the new database. Each createCollection call returns { ok: 1 }, and show collections then lists the five names in alphabetical order, confirming the database exists on disk with all five collections empty.

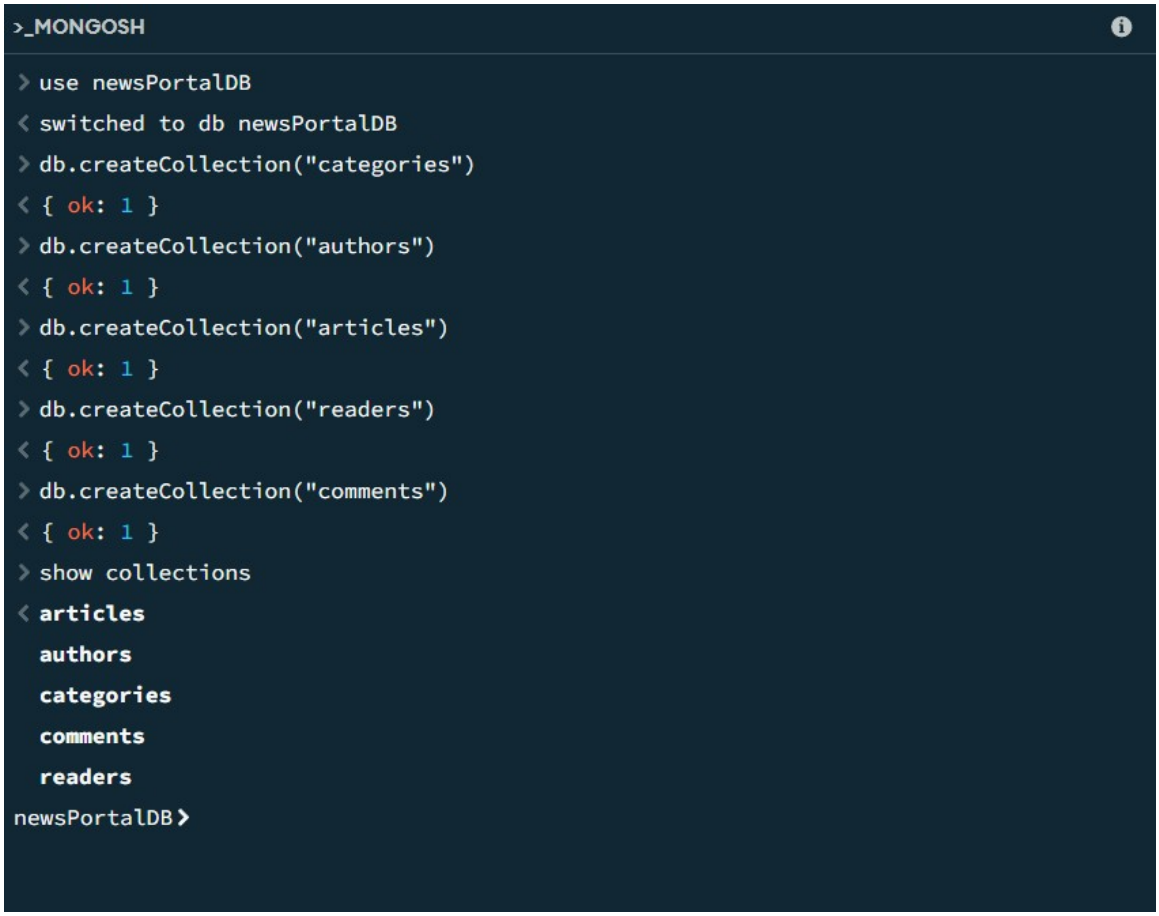


Fig. 1 — Database and collections created, verified with show collections

5. Create operations

The case study asks for eight to ten documents in each major collection. The counts used here are eight categories, eight authors, ten articles, eight readers and ten comments. The data was chosen so that later queries have something to find: one category is inactive, one article is a draft with a null publish date, two authors are unverified, four readers are on the free plan, and two comments are spam.

5.1 insertOne() on categories

```
db.categories.insertOne({
  _id: "CAT01",
  name: "Politics",
  slug: "politics",
  description: "National and international political news",
  isActive: true,
  createdAt: new Date("2025-01-10")
})
```

A single document inserted to demonstrate insertOne. The shell replies with acknowledged: true and the insertedId, which is CAT01 rather than a generated ObjectId because _id was supplied explicitly. new Date() stores a real BSON date, not a string, so the date comparisons in R5 work later.



Fig. 2 — insertOne() adding the Politics category

5.2 insertMany() on categories

```
db.categories.insertMany([
  { _id: "CAT02", name: "Technology", slug: "technology", ... },
  { _id: "CAT03", name: "Sports", slug: "sports", ... },
  ...
```

```
  { _id: "CAT08", name: "Education", slug: "education", isActive: false, ... }
])
```

The remaining seven categories go in as one array. insertMany returns all seven insertedIds keyed by their position in the array. CAT08 Education is given isActive: false so that the deleteMany in Task 8 has a target.

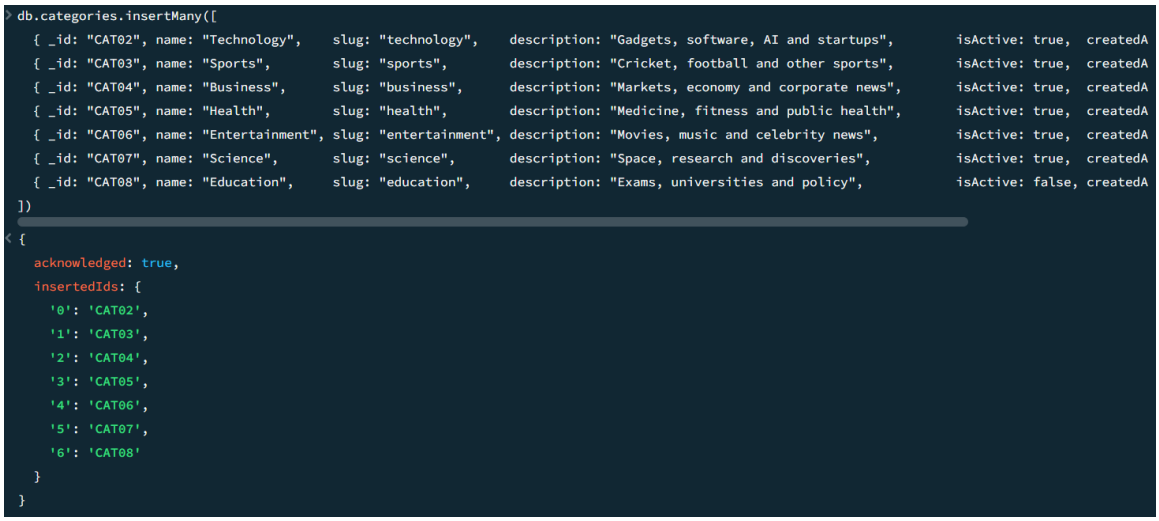


Fig. 3 — insertMany() adding the remaining seven categories

5.3 insertMany() on authors

```
db.authors.insertMany([
  { _id: "AUT01", name: "Priya Raman", expertise: ["Politics", "Elections"],
    social: { twitter: "@priyaraman", linkedin: "in/priyaraman" },
    isVerified: true, joinedOn: new Date("2022-06-15"), articleCount: 142 },
  ...
])
```

Eight author profiles. Two things are worth noting in the output. The expertise field is an array, so an author can be matched on any one of their beats with a plain equality query. The social field is an embedded sub-document, and some of its values are null, which is how an author with no LinkedIn profile is stored without inventing a placeholder.

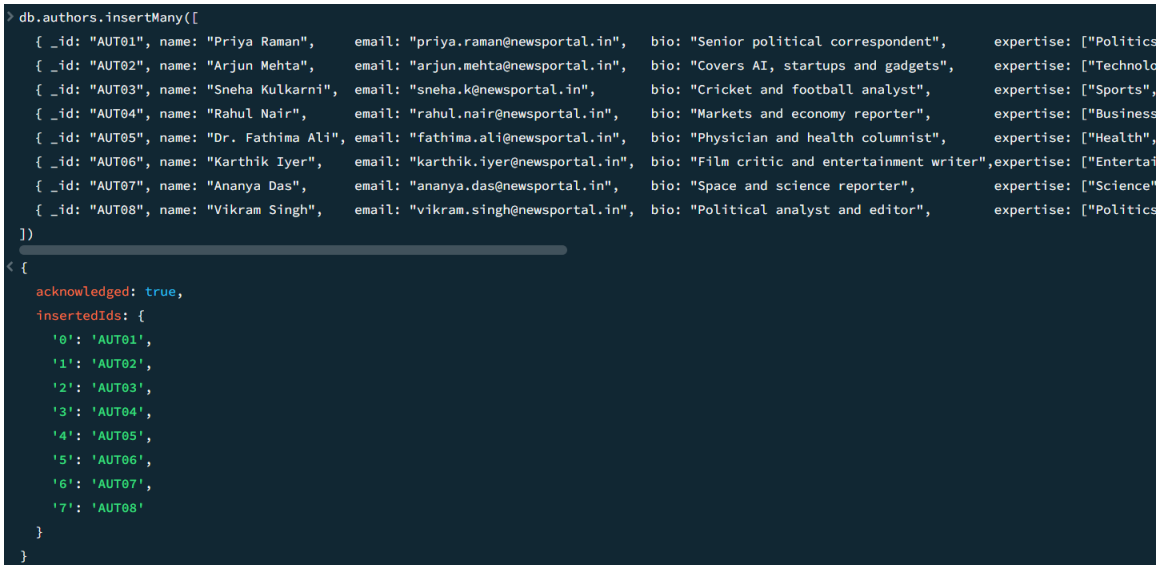


Fig. 4 — insertMany() adding eight author profiles

5.4 insertMany() on articles

```
db.articles.insertMany([
  { _id: "ART01", title: "India Announces New AI Policy Framework",
    authorId: "AUT02", categoryId: "CAT02",
    tags: ["AI", "policy", "India"], status: "published",
    publishedAt: new Date("2026-10-02"), views: 15420, likes: 980,
    media: { coverImage: "ai_policy.jpg", caption: "Delegates at the policy
launch" },
    location: { city: "New Delhi", country: "India" } },
  ...
])
```

Ten articles, and the collection where the design decisions from Task 3 are visible together. authorId and categoryId are references holding codes from the other two collections. tags is an array. media and location are embedded sub-documents. ART10 is inserted as a draft with publishedAt set to null, views and likes at zero, and a null coverImage, which is the state an unfinished story is in before an editor publishes it.

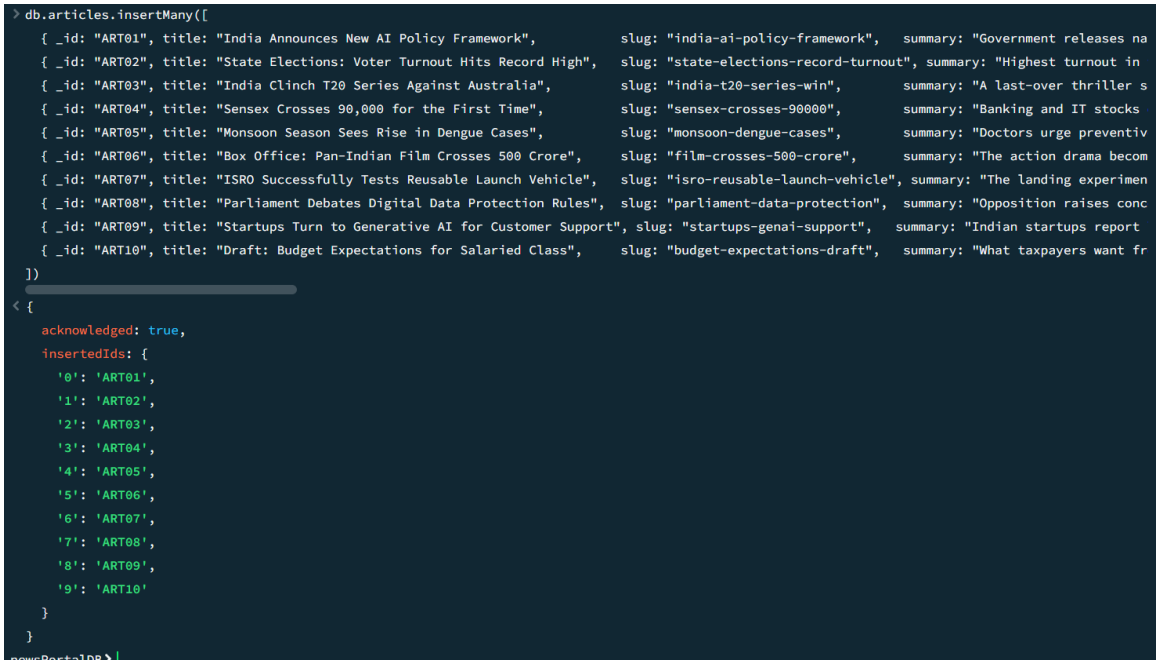


Fig. 5 — insertMany() adding ten articles

5.5 insertMany() on readers

```
db.readers.insertMany([
  { _id: "RDR01", name: "Aditya Sharma", city: "Chennai",
    preferences: { categories: ["CAT02", "CAT03"], newsletter: true },
    subscription: { plan: "premium", startDate: new Date("2026-04-01"),
                    endDate: new Date("2026-12-31") },
    bookmarks: ["ART01", "ART03"] },
  ...
])
```

Eight readers. Each one carries two embedded documents, preferences and subscription, and an array of article codes under bookmarks. Four readers are premium and four are free; on a free plan endDate is null. RDR04 has an empty bookmarks array, which is a different thing from the field being absent and is worth keeping consistent.

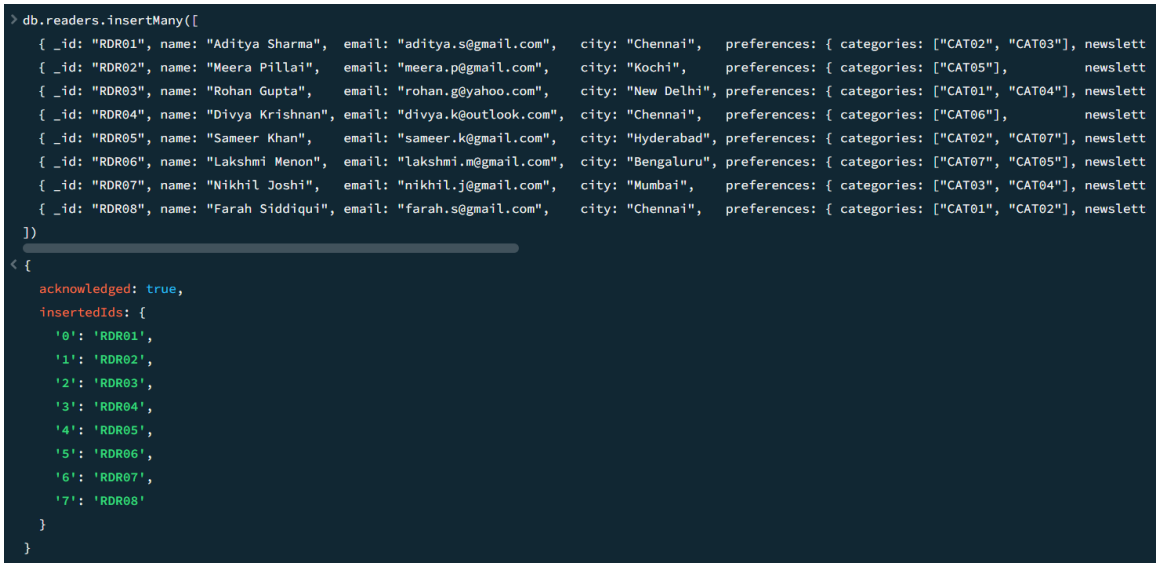


Fig. 6 — insertMany() adding eight reader accounts

5.6 insertMany() on comments

```
db.comments.insertMany([
  { _id: "CMT01", articleId: "ART03", readerId: "RDR01",
    text: "What a finish! Best match of the year.",
    postedAt: new Date("2026-10-08T21:10:00"), likes: 54, isFlagged: false,
    replies: [ { readerId: "RDR07", text: "Agreed, that last over was unreal.",
                 postedAt: new Date("2026-10-08T21:30:00") } ] },
  ...
])
```

Ten comments. Each one references an article and a reader by code and embeds its replies as an array of sub-documents, so a whole thread comes back in one read. Three comments carry a reply; the rest have an empty array. CMT07 and CMT10 are flagged spam, which is what the Task 8 deletes act on. The timestamps include a time of day, so comments under the same article can be ordered correctly.

```
db.comments.insertMany([
  { _id: "CMT01", articleId: "ART03", readerId: "RDR01", text: "What a finish! Best match of the year.",           postedAt: new Date("2026-10-08T
  { _id: "CMT02", articleId: "ART01", readerId: "RDR05", text: "Good step, but enforcement will be the real test.", postedAt: new Date("2026-10-02T
  { _id: "CMT03", articleId: "ART02", readerId: "RDR03", text: "Great to see youth participation rising.",         postedAt: new Date("2026-10-05T
  { _id: "CMT04", articleId: "ART07", readerId: "RDR06", text: "Proud moment for Indian science!",                 postedAt: new Date("2026-10-14T
  { _id: "CMT05", articleId: "ART05", readerId: "RDR02", text: "Our area hasn't had fogging in weeks.",            postedAt: new Date("2026-09-19T
  { _id: "CMT06", articleId: "ART06", readerId: "RDR04", text: "Overrated movie honestly.",                        postedAt: new Date("2026-10-12T
  { _id: "CMT07", articleId: "ART06", readerId: "RDR08", text: "CLICK HERE FOR FREE MOVIE DOWNLOADS!!!",           postedAt: new Date("2026-10-13T
  { _id: "CMT08", articleId: "ART04", readerId: "RDR07", text: "Is this rally sustainable?",                       postedAt: new Date("2026-09-26T
  { _id: "CMT09", articleId: "ART08", readerId: "RDR03", text: "Consent rules need to be much clearer.",           postedAt: new Date("2026-10-20T
  { _id: "CMT10", articleId: "ART03", readerId: "RDR04", text: "Buy cheap followers at spam-link dot com",         postedAt: new Date("2026-10-09T
])
{
  acknowledged: true,
  insertedIds: {
    '0': 'CMT01',
    '1': 'CMT02',
    '2': 'CMT03',
    '3': 'CMT04',
    '4': 'CMT05',
    '5': 'CMT06',
    '6': 'CMT07',
    '7': 'CMT08',
    '8': 'CMT09',
    '9': 'CMT10'
  }
}
```

Fig. 7 — insertMany() adding ten comments with embedded replies

Verifying the inserted counts

```
db.categories.countDocuments()
db.authors.countDocuments()
db.articles.countDocuments()
db.readers.countDocuments()
db.comments.countDocuments()
```

countDocuments runs on each collection to confirm the inserts landed. The expected output is 8, 8, 10, 8 and 10.

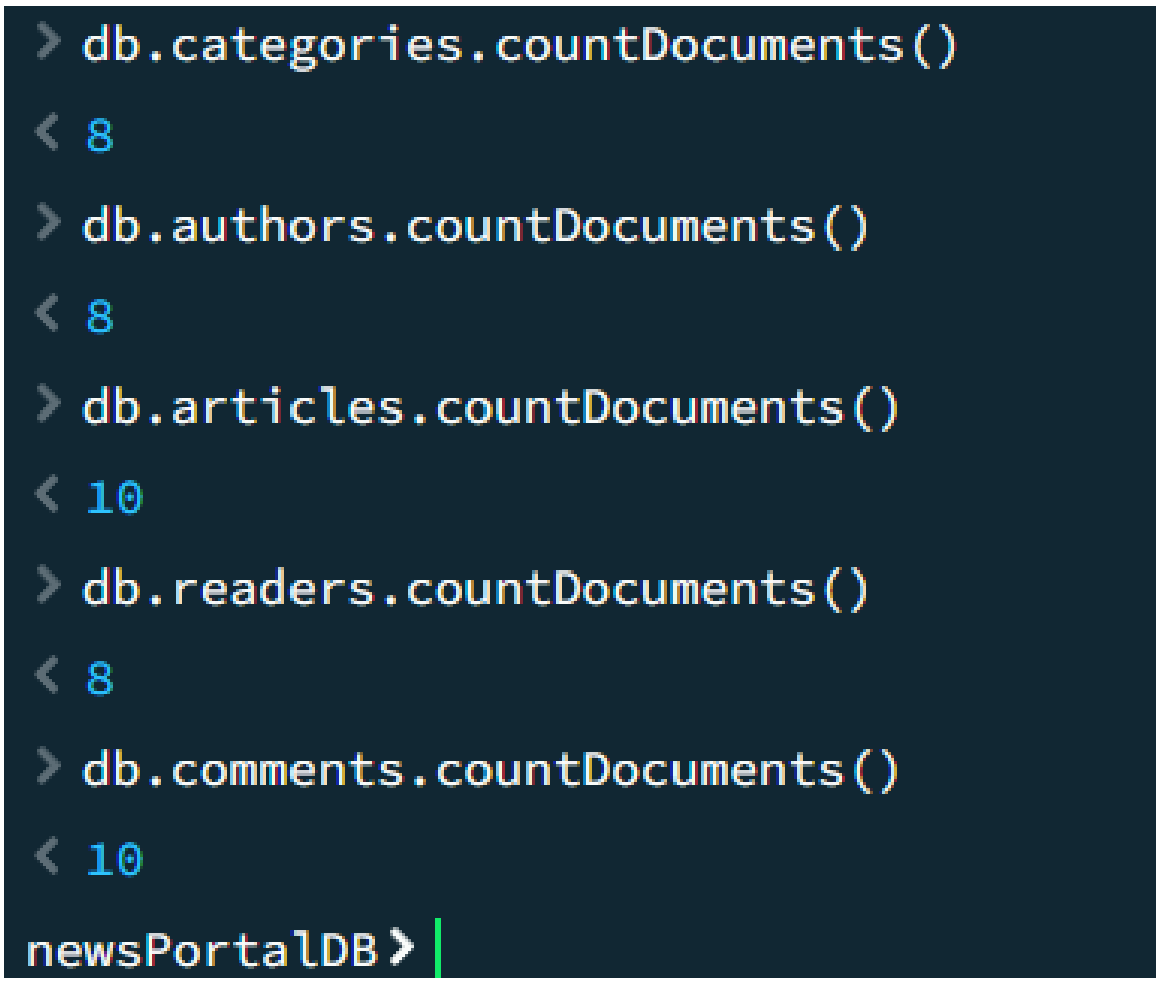


Fig. 8 — countDocuments() confirming 8, 8, 10, 8 and 10 documents

6. Read operations

Twelve read operations are shown, against a required minimum of eight. The first ten use find() with filters, projections, sorting and limits; the last two use the aggregation pipeline to join references and to group. Every query is one the portal would actually run, so each is introduced by the page or feature it belongs to.

A note on projections: the second argument to find() lists the fields to return. _id: 0 suppresses the identifier, which is what the site does when it only needs display data. Where _id is useful for reading the output it is left in.

R1. Filter, projection and sort together

```
db.articles.find(
  { categoryId: "CAT02", status: "published" },
  { _id: 0, title: 1, views: 1, publishedAt: 1 }
).sort({ views: -1 })
```

This builds the Technology section page. The filter matches on two fields at once, which MongoDB treats as an implicit AND, so only published Technology articles come back. The projection cuts the body text out of the result, since a section page shows headlines. sort({ views: -1 }) orders the result most-read first.

```
db.articles.find(
  { categoryId: "CAT02", status: "published" },
  { _id: 0, title: 1, views: 1, publishedAt: 1 }
).sort({ views: -1 })
[
  {
    title: 'India Announces New AI Policy Framework',
    publishedAt: ISODate('2026-10-02T00:00:00.000Z'),
    views: 15420
  }
  {
    title: 'Startups Turn to Generative AI for Customer Support',
    publishedAt: ISODate('2026-10-22T00:00:00.000Z'),
    views: 2980
  }
```

Fig. 9 — R1: published Technology articles sorted by view count

R2. Comparison operators

```
db.articles.find(
  { views: { $gt: 10000 }, likes: { $gte: 1000 } },
  { _id: 0, title: 1, views: 1, likes: 1 }
)
```

The editor's definition of a hit: more than ten thousand views and at least a thousand likes. $gt is strictly greater than and $gte includes the boundary, so an article sitting on exactly 1000 likes still qualifies. Both conditions have to hold, so ART01 with 980 likes is excluded despite clearing the view threshold.

```
> db.articles.find(
    { views: { $gt: 10000 }, likes: { $gte: 1000 } },
    { _id: 0, title: 1, views: 1, likes: 1 }
  )
< {
    title: 'State Elections: Voter Turnout Hits Record High',
    views: 22310,
    likes: 1450
  }
  {
    title: 'India Clinch T20 Series Against Australia',
    views: 31050,
    likes: 2600
  }
  {
    title: 'ISRO Successfully Tests Reusable Launch Vehicle',
    views: 18760,
    likes: 1720
  }
newsPortalDB>
```

Fig. 10 — R2: articles above 10,000 views and 1,000 likes

R3. Querying inside an array with $in

```
db.articles.find(
  { tags: { $in: ["AI", "elections"] } },
  { _id: 0, title: 1, tags: 1 }
)
```

The related-stories box. tags is an array, and MongoDB matches an array field if any element matches, so no special syntax is needed to look inside it. $in widens that to a set of acceptable values, returning any article tagged either AI or elections.



Fig. 11 — R3: articles tagged AI or elections

R4. Sort with limit

```
db.articles.find({}, { _id: 0, title: 1, views: 1 })
  .sort({ views: -1 })
  .limit(3)
```

The trending box on the homepage. An empty filter document matches everything, the sort puts the most-viewed first, and limit(3) stops after three. MongoDB applies sort before limit regardless of the order the methods are chained in, so this really does return the top three and not three arbitrary documents.

```
> db.articles.find({}, { _id: 0, title: 1, views: 1 })
    .sort({ views: -1 })
    .limit(3)
< {
    title: 'India Clinch T20 Series Against Australia',
    views: 31050
  }
  {
    title: 'State Elections: Voter Turnout Hits Record High',
    views: 22310
  }
  {
    title: 'ISRO Successfully Tests Reusable Launch Vehicle',
    views: 18760
  }
```

Fig. 12 — R4: top three most-read articles

R5. Date range query

```
db.articles.find(
  { publishedAt: { $gte: new Date("2026-10-01"), $lte: new Date("2026-10-15") } },
  { _id: 0, title: 1, publishedAt: 1 }
).sort({ publishedAt: 1 })
```

The archive page for the first half of October. Both bounds are applied to the same field inside one object, which is how a range is expressed. This only works because publishedAt was stored with new Date() at insert time; had the dates been stored as strings the comparison would have been alphabetical. The draft article is skipped automatically, since null is not within the range.

```
> db.articles.find(
    { publishedAt: { $gte: new Date("2026-10-01"), $lte: new Date("2026-10-15") } },
    { _id: 0, title: 1, publishedAt: 1 }
  ).sort({ publishedAt: 1 })
< {
    title: 'India Announces New AI Policy Framework',
    publishedAt: ISODate('2026-10-02T00:00:00.000Z')
  }
  {
    title: 'State Elections: Voter Turnout Hits Record High',
    publishedAt: ISODate('2026-10-05T00:00:00.000Z')
  }
  {
    title: 'India Clinch T20 Series Against Australia',
    publishedAt: ISODate('2026-10-08T00:00:00.000Z')
  }
  {
    title: 'Box Office: Pan-Indian Film Crosses 500 Crore',
    publishedAt: ISODate('2026-10-11T00:00:00.000Z')
  }
  {
    title: 'ISRO Successfully Tests Reusable Launch Vehicle',
    publishedAt: ISODate('2026-10-14T00:00:00.000Z')
  }
```

Fig. 13 — R5: articles published between 1 and 15 October 2026

R6. Querying embedded fields with dot notation

```
db.readers.find(
  { "subscription.plan": "premium", "subscription.endDate": { $lt: new Date("2027-
01-01") } },
  { _id: 0, name: 1, email: 1, "subscription.endDate": 1 }
)
```

The renewal reminder list. subscription is an embedded document, and dot notation reaches into it without the whole sub-document having to be matched. The quotes around the key are required because of the dot. The projection uses dot notation as well, returning only the end date out of the subscription block. Two readers come back rather than three because this was captured after the Task 7 subscription job had run: RDR07's plan was moved to free by U6 and no longer satisfies the premium condition.

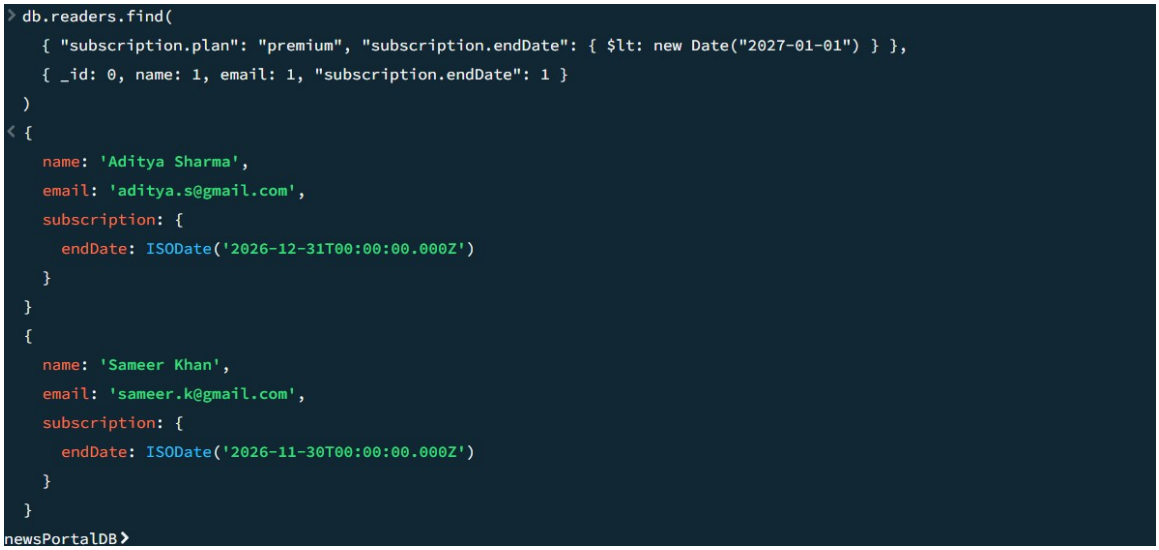


Fig. 14 — R6: premium readers whose subscription expires before 2027

R7. Array match with a boolean filter

```
db.authors.find(
  { expertise: "Politics", isVerified: true },
  { _id: 0, name: 1, expertise: 1, articleCount: 1 }
)
```

Choosing a writer for a political story. expertise: "Politics" matches any author whose expertise array contains that value, so AUT08 Vikram Singh, whose array is ["Politics", "Business"], is returned alongside AUT01. The boolean condition then narrows it to verified profiles.

```
> db.authors.find(
    { expertise: "Politics", isVerified: true },
    { _id: 0, name: 1, expertise: 1, articleCount: 1 }
  )
< {
    name: 'Priya Raman',
    expertise: [
      'Politics',
      'Elections'
    ],
    articleCount: 142
  }
  {
    name: 'Vikram Singh',
    expertise: [
      'Politics',
      'Business'
    ],
    articleCount: 305
  }
```

Fig. 15 — R7: verified authors who cover Politics

R8. Case-insensitive search with a regular expression

```
db.articles.find(
  { title: { $regex: "india", $options: "i" } },
  { _id: 0, title: 1 }
)
```

The site search box. $regex matches a substring anywhere in the title, and the i option makes it case-insensitive, so the lowercase search term finds titles containing India. The third result is worth looking at: the box office headline matches on Pan-Indian, because $regex looks for the letters anywhere in the string and does not respect word boundaries. A reader searching for India gets a film story back. This is a convenience rather than a fast query: without a text index MongoDB scans every document, which is fine at ten articles and would not be at ten million.

```
> db.articles.find(
    { title: { $regex: "india", $options: "i" } },
    { _id: 0, title: 1 }
  )
< {
    title: 'India Announces New AI Policy Framework'
  }
  {
    title: 'India Clinch T20 Series Against Australia'
  }
  {
    title: 'Box Office: Pan-Indian Film Crosses 500 Crore'
  }
```

Fig. 16 — R8: title search for india, case-insensitive

R9. $or combined with array position

```
db.comments.find(
  { $or: [ { isFlagged: true }, { "replies.0": { $exists: true } } ] },
  { _id: 1, text: 1, isFlagged: 1, replies: 1 }
)
```

The moderation queue, which wants both spam and any comment that has started a discussion. $or takes an array of conditions and matches a document satisfying at least one. The second condition is the interesting one: replies.0 addresses the first element of the array by index, and $exists: true is true only when that element is present, which is a direct test for a non-empty array.

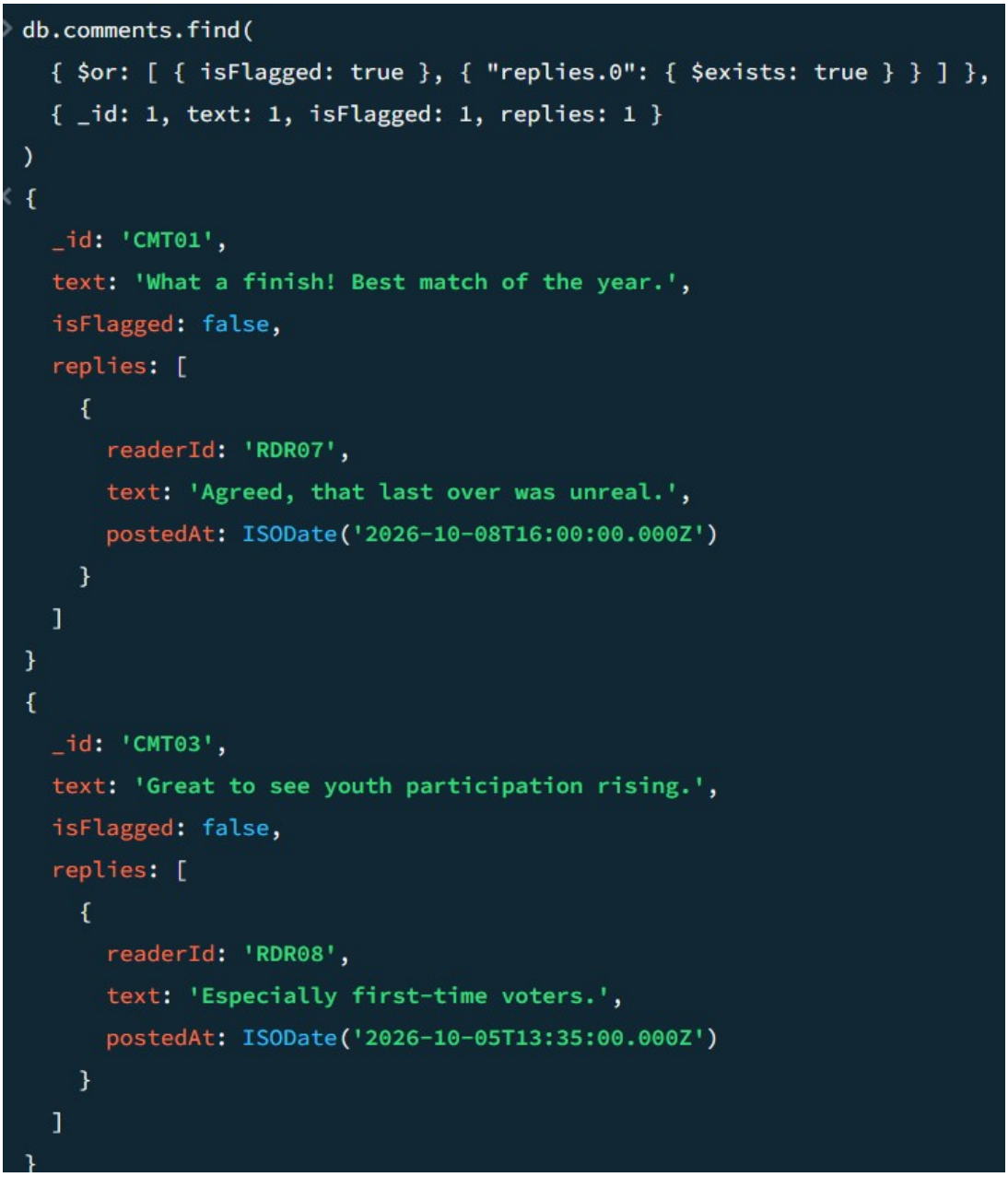


Fig. 17 — R9: flagged comments or comments with at least one reply

R10. Counting matching documents

```
db.articles.countDocuments({ status: "draft" })
```

How many stories are sitting unfinished, shown on the editor's dashboard. countDocuments takes the same filter syntax as find but returns a number rather than a cursor. The answer is 1, which is ART10.

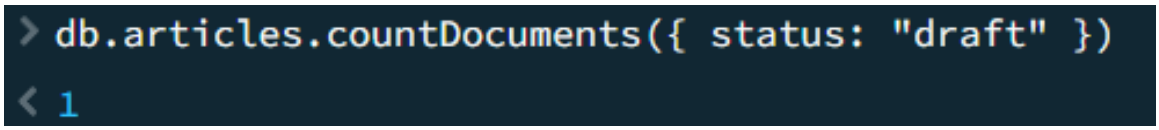


Fig. 18 — R10: count of draft articles

R11. Aggregation with $lookup to resolve references

```
db.articles.aggregate([
  { $match: { status: "published" } },
  { $lookup: { from: "authors",    localField: "authorId",   foreignField: "_id",
as: "author" } },
  { $lookup: { from: "categories", localField: "categoryId", foreignField: "_id",
as: "category" } },
  { $project: { _id: 0, title: 1,
      author:   { $arrayElemAt: ["$author.name", 0] },
      category: { $arrayElemAt: ["$category.name", 0] },
      views: 1 } },
  { $sort: { views: -1 } }
])
```

This is the price of referencing instead of embedding, and the thing that makes it worth paying. The pipeline runs in order. $match filters to published articles first, so the joins have less to do. Each $lookup is MongoDB's left outer join: it reads the named collection, matches localField against foreignField and attaches the result as an array. Because an _id match returns at most one document, $arrayElemAt with index 0 pulls the single name out of that array so the output reads as a flat row rather than a nested array. $project then shapes the output and $sort orders it.

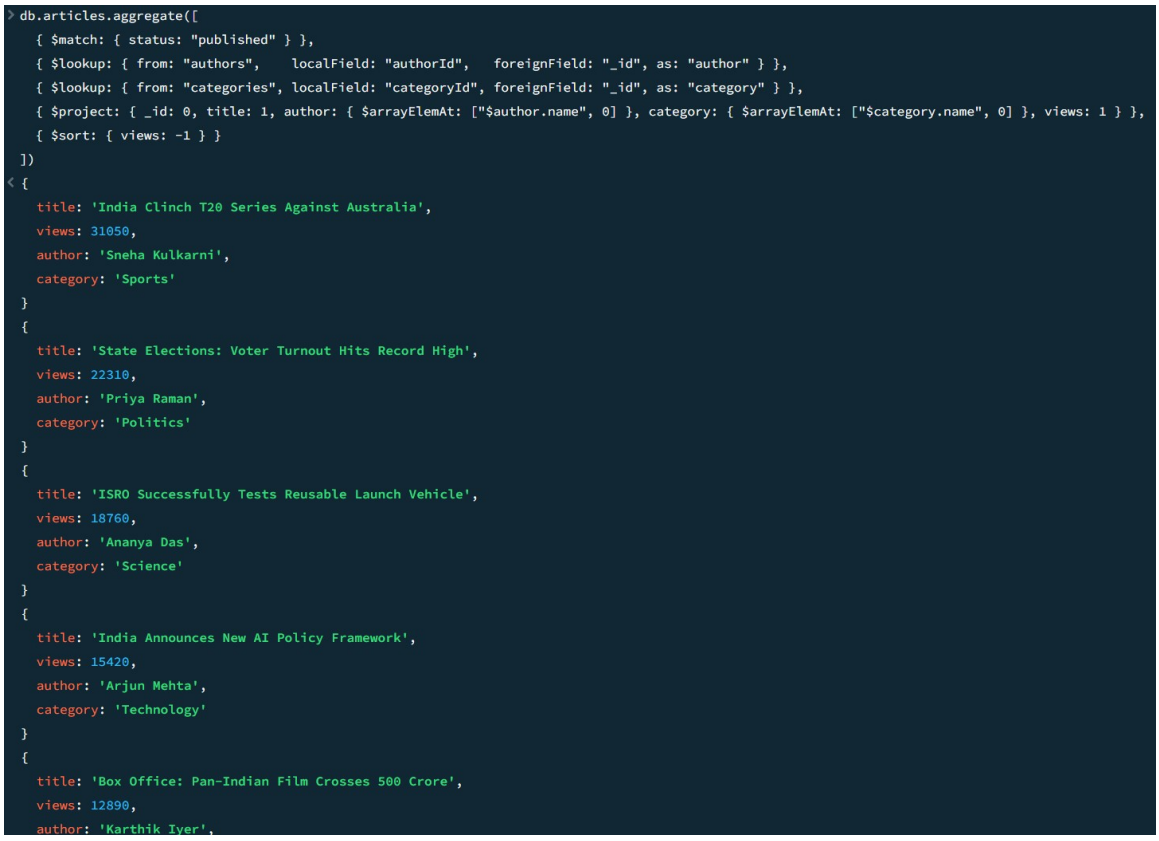


Fig. 19 — R11: articles joined to their author and category names

R12. Aggregation with $group

```
db.articles.aggregate([
  { $group: { _id: "$categoryId", totalArticles: { $sum: 1 }, totalViews: { $sum:
"$views" } } },
  { $lookup: { from: "categories", localField: "_id", foreignField: "_id", as: "cat"
} },
  { $project: { _id: 0, category: { $arrayElemAt: ["$cat.name", 0] }, totalArticles:
1, totalViews: 1 } },
  { $sort: { totalViews: -1 } }
])
```

The traffic report the editorial team looks at weekly. $group collapses documents that share a value: here _id: "$categoryId" makes one output document per category. Inside the group, $sum: 1 adds one for each article and so counts them, while $sum: "$views" adds up the actual view numbers. The dollar sign in front of a field name means read this document's value, which is what separates it from the literal 1. The category name is then looked up, since grouping leaves only the code behind.

```
> db.articles.aggregate([
    { $group: { _id: "$categoryId", totalArticles: { $sum: 1 }, totalViews: { $sum: "$views" } } },
    { $lookup: { from: "categories", localField: "_id", foreignField: "_id", as: "cat" } },
    { $project: { _id: 0, category: { $arrayElemAt: ["$cat.name", 0] }, totalArticles: 1, totalViews: 1 } },
    { $sort: { totalViews: -1 } }
  ])
< {
    totalArticles: 1,
    totalViews: 31050,
    category: 'Sports'
  }
  {
    totalArticles: 2,
    totalViews: 26620,
    category: 'Politics'
  }
  {
    totalArticles: 1,
    totalViews: 18760,
    category: 'Science'
  }
  {
    totalArticles: 2,
    totalViews: 18400,
    category: 'Technology'
  }
  {
    totalArticles: 1,
    totalViews: 12890,
    category: 'Entertainment'
  }
  {
    totalArticles: 2,
    totalViews: 8740,
    category: 'Business'
```

Fig. 20 — R12: article count and total views per category

7. Update operations

Six updates are shown against a required minimum of four. Each one runs the update first and then a find on the affected document, so the screenshot shows both the acknowledgement and the changed data. The acknowledgement reports matchedCount and modifiedCount separately, and the two differ when a document matches the filter but already holds the value being set.

U1. updateOne() with $inc

```
db.articles.updateOne(
  { _id: "ART09" },
  { $inc: { views: 1, likes: 1 } }
)
db.articles.find({ _id: "ART09" }, { title: 1, views: 1, likes: 1 })
```

A reader opens ART09 and likes it. $inc adds to the existing value on the server without the application having to read it first, which avoids the lost-update problem two readers hitting the same article at the same moment would otherwise cause. Views go from 2980 to 2981 and likes from 140 to 141.

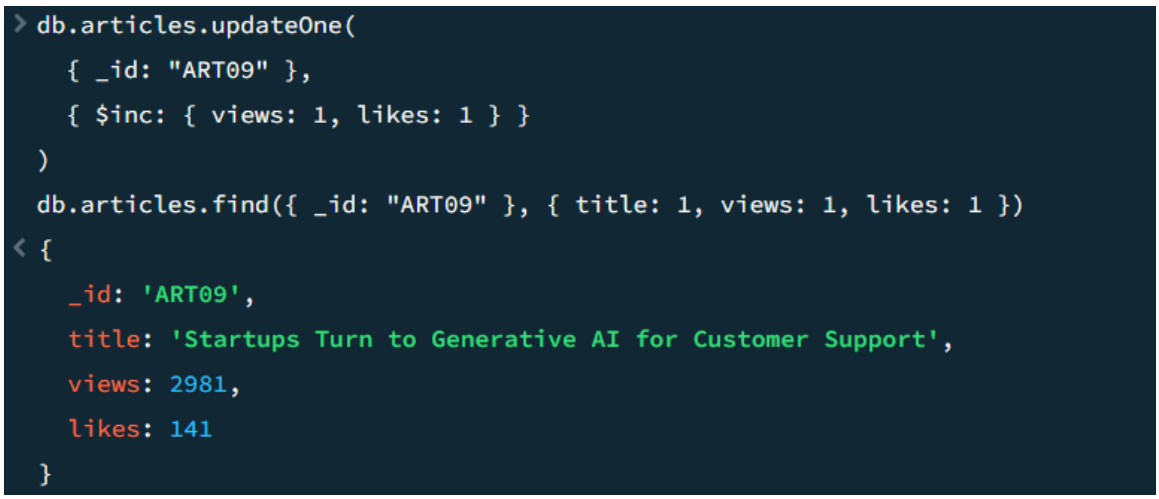


Fig. 21 — U1: $inc raising views and likes on ART09

U2. updateMany() with $set

```
db.articles.updateMany(
  { views: { $gt: 15000 } },
  { $set: { isFeatured: true } }
)
db.articles.find({ isFeatured: true }, { _id: 0, title: 1, views: 1, isFeatured:
1 })
```

A nightly job promoting anything past fifteen thousand views to the featured strip. updateMany applies the change to every matching document, unlike updateOne which stops at the first. Four articles clear the threshold: ART01 on 15,420 views, ART02 on 22,310, ART03 on 31,050 and ART07 on 18,760. ART03 was already featured when it was inserted, so matchedCount is 4 while modifiedCount is 3. That gap is the clearest illustration of why MongoDB reports the two numbers separately: a document can satisfy the filter and still need no change.

```
> db.articles.updateMany(
    { views: { $gt: 15000 } },
    { $set: { isFeatured: true } }
  )
  db.articles.find({ isFeatured: true }, { _id: 0, title: 1, views: 1, isFeatured: 1 })
< {
    title: 'India Announces New AI Policy Framework',
    views: 15420,
    isFeatured: true
  }
  {
    title: 'State Elections: Voter Turnout Hits Record High',
    views: 22310,
    isFeatured: true
  }
  {
    title: 'India Clinch T20 Series Against Australia',
    views: 31050,
    isFeatured: true
  }
  {
    title: 'ISRO Successfully Tests Reusable Launch Vehicle',
    views: 18760,
    isFeatured: true
  }
```

Fig. 22 — U2: updateMany() featuring every article above 15,000 views

U3. updateOne() with $set and $currentDate

```
db.articles.updateOne(
  { _id: "ART10" },
  { $set: { status: "published", title: "Budget 2027: What the Salaried Class
Expects" },
    $currentDate: { publishedAt: true } }
)
db.articles.find({ _id: "ART10" }, { title: 1, status: 1, publishedAt: 1 })
```

An editor publishes the draft budget story. Two operators are used in one call: $set replaces the status and the working title, and $currentDate writes the server's clock into publishedAt. Taking the timestamp from the server rather than from the client means every article is dated against the same clock no matter which machine published it. After this the draft count from R10 drops to zero.

```
> db.articles.updateOne(
    { _id: "ART10" },
    { $set: { status: "published", title: "Budget 2027: What the Salaried Class Expects" },
      $currentDate: { publishedAt: true } }
  )
  db.articles.find({ _id: "ART10" }, { title: 1, status: 1, publishedAt: 1 })
< {
    _id: 'ART10',
    title: 'Budget 2027: What the Salaried Class Expects',
    status: 'published',
    publishedAt: ISODate('2026-10-25T07:33:59.874Z')
  }
```

Fig. 23 — U3: publishing the draft article with $set and $currentDate

U4. updateOne() with $addToSet

```
db.articles.updateOne(
  { _id: "ART07" },
  { $addToSet: { tags: "technology" } }
)
db.articles.find({ _id: "ART07" }, { title: 1, tags: 1 })
```

Adding a tag to the ISRO story. $addToSet appends to the array only when the value is not already in it, where $push would append unconditionally and leave a duplicate behind. Running this a second time matches the document but modifies nothing, which the acknowledgement shows.

```
db.articles.updateOne(
  { _id: "ART07" },
  { $addToSet: { tags: "technology" } }
)
db.articles.find({ _id: "ART07" }, { title: 1, tags: 1 })
{
  _id: 'ART07',
  title: 'ISRO Successfully Tests Reusable Launch Vehicle',
  tags: [
    'ISRO',
    'space',
    'India',
    'technology'
  ]
}
```

Fig. 24 — U4: $addToSet adding a tag without duplicating it

U5. updateOne() with $push into an embedded array

```
db.comments.updateOne(
  { _id: "CMT02" },
  { $push: { replies: { readerId: "RDR03",
                        text: "True, rules mean nothing without audits.",
                        postedAt: new Date() } } }
)
db.comments.find({ _id: "CMT02" })
```

A reader replies to CMT02. $push is correct here rather than $addToSet, because two readers may legitimately post the same text and neither should be silently dropped. The value pushed is a whole sub-document, so the embedded reply structure from Task 3 is preserved. CMT02 went in with an empty replies array and now holds one element.



Fig. 25 — U5: $push adding a reply inside a comment document

U6. updateMany() on an embedded field

```
db.readers.updateMany(
  { "subscription.plan": "premium", "subscription.endDate": { $lt: new Date() } },
  { $set: { "subscription.plan": "free", "subscription.endDate": null } }
)
db.readers.find({}, { _id: 0, name: 1, "subscription.plan": 1,
"subscription.endDate": 1 })
```

The subscription expiry job. The filter uses dot notation to reach into the embedded subscription document, and $set uses the same notation to change one field inside it without overwriting the rest of the block. Passing the whole sub-document instead would wipe out startDate. new Date() with no argument is the moment the command runs, so RDR07, whose premium plan ended on 9 October 2026, is moved to free.

```
db.readers.updateMany(
  { "subscription.plan": "premium", "subscription.endDate": { $lt: new Date() } },
  { $set: { "subscription.plan": "free", "subscription.endDate": null } }
)
db.readers.find({}, { _id: 0, name: 1, "subscription.plan": 1, "subscription.endDate": 1 })
{
  name: 'Aditya Sharma',
  subscription: {
    plan: 'premium',
    endDate: ISODate('2026-12-31T00:00:00.000Z')
  }
}
{
  name: 'Meera Pillai',
  subscription: {
    plan: 'free',
    endDate: null
  }
}
{
  name: 'Rohan Gupta',
  subscription: {
    plan: 'premium',
    endDate: ISODate('2027-01-14T00:00:00.000Z')
  }
}
{
  name: 'Divya Krishnan',
  subscription: {
    plan: 'free',
    endDate: null
  }
}
```

Fig. 26 — U6: expired premium subscriptions downgraded to free

8. Delete operations

Three deletes are shown against a required minimum of two. Delete operations are irreversible in MongoDB, so each filter here was run as a find first to confirm it matched only what was intended.

D1. deleteOne()

```
db.comments.deleteOne({ _id: "CMT07" })
db.comments.countDocuments()
```

A moderator removes one spam comment, the fake download link posted under the box office story. deleteOne removes at most one document even when the filter matches several, which is the safe

default for a moderator acting on a specific comment. The acknowledgement reports deletedCount: 1 and the collection count drops from 10 to 9.

```
> db.comments.deleteOne({ _id: "CMT07" })
  db.comments.countDocuments()
< 9
```

Fig. 27 — D1: deleteOne() removing a single spam comment

D2. deleteMany()

```
db.comments.deleteMany({ isFlagged: true })
db.comments.find({}, { _id: 1, text: 1, isFlagged: 1 })
```

Clearing the rest of the spam queue in one operation. CMT07 is already gone, so only CMT10 remains flagged and deletedCount is 1. The find afterwards shows eight comments left, none of them flagged. Running the same command again would delete nothing and still report success, since a delete that matches no document is not an error.



Fig. 28 — D2: deleteMany() clearing the remaining flagged comments

D3. deleteMany() on categories

```
db.categories.deleteMany({ isActive: false })
db.categories.find({}, { _id: 1, name: 1, isActive: 1 })
```

Retiring the Education desk, which was inserted as inactive and has no articles filed against it. The find afterwards shows seven categories remaining. This one is worth a caveat: MongoDB does not enforce foreign keys, so deleting a category that still had articles would leave those articles pointing at a categoryId that no longer resolves, and the $lookup in R11 would return an empty array for them. The isActive flag is checked by hand here because the database will not check it for you.



Fig. 29 — D3: deleteMany() removing the inactive category

Summary of operations performed

Task	Required	Performed
Collections	5 minimum	5 — categories, authors, articles, readers, comments
Create	8-10 documents per major collection	8 categories, 8 authors, 10 articles, 8 readers, 10 comments
Read	8 minimum	12 — R1 to R12, including two

		aggregation pipelines
`Update`	4 minimum	6 — U1 to U6, using $inc, $set, $currentDate, $addToSet and $push
`Delete`	2 minimum	3 — one deleteOne and two deleteMany